NATIVE n8n AUTOMATION SERVER

Complete Production Sizing, Deployment & Operations Manual

ENTERPRISE SPECIFICATION - LINUX SYSTEMS (CROSS-DISTRO)

Target Platform:	Ubuntu / Debian / RHEL / Fedora / Arch Linux
Document Type:	System Configuration & Verification Guide
Script Version:	v3.0 (Production-Ready Edition)
Author:	b1swa (sandipbiswa10@gmail.com)
Created Date:	May 30, 2026
Classification:	Internal Technical Reference

Table of Contents & Version Evolution

1. Version History & Script Evolution

The Native n8n Setup Suite has evolved through three iterations to address security, networking, and production reliability concerns.

- **Version 1.0:** Baseline setup. Captured parameters (port, webhook) via whiptail, installed Node.js, and started n8n under PM2. Included no support for custom databases or secure credentials keys.
- **Version 2.0:** Introduced custom network configuration (Static IP via Netplan/nmcli), secure .env management at /etc/n8n/.env with 600 permissions, automated daily backups with retention, and helper scripts.
- **Version 3.0:** Added automated Nginx reverse proxy configurations with WebSocket headers, Certbot SSL automation, PM2 Log Rotation (pm2-logrotate), custom Node memory limits, and a clean uninstall script.

2. Concept Dictionary (Plain English Analogies)

To make this deployment manual accessible to everyone, we explain core technical concepts using everyday analogies:

- **Server Port (e.g. 5678):** Imagine your server computer is a large apartment building. The 'Port' is like an individual apartment number. n8n lives inside apartment number 5678, waiting to receive visitors.

- **Nginx Proxy:** Think of Nginx as a friendly receptionist sitting in the building's lobby. When external users visit your domain, Nginx greets them at the main entrance (Port 80/443) and safely guides them back to n8n's apartment (Port 5678).

- **Database Backend:** This is n8n's digital filing cabinet. SQLite is a small cabinet in the room (good for small test files), whereas PostgreSQL is a secure filing room down in the basement (great for millions of files).

- **PM2 Daemon Manager:** Think of PM2 as a security guard who watches n8n 24 hours a day. If n8n trips and falls down (crashes), the guard immediately dusts it off and helps it stand back up (restarts it).

- **Let's Encrypt SSL:** This is like a lockable security envelope. Instead of sending messages on post-cards that anyone can read, SSL wraps all network traffic inside encrypted envelopes.

3. Architecture Overview & System Prerequisites

A 'Native' installation executes n8n directly on your server's Linux operating system without virtual barriers. This reduces RAM and CPU overhead, allowing files and workflow webhooks to run faster.

Minimum Hardware & System Requirements

- **CPU (Processor):** Minimum 1 processor core. Like a brain, more cores allow n8n to think about and run multiple automation tasks at the same time.
- **RAM (Memory):** Minimum 1 Gigabyte (GB) of RAM. If you choose a PostgreSQL database, 2 GB is highly recommended so both programs have plenty of room to work.
- **Linux Distributions:** Works on Ubuntu 20.04+, Debian 11+, Fedora 38+, CentOS Stream 9, Red Hat Enterprise Linux 8/9, or Arch Linux.
- **Root Access:** Administrator rights (sudo privileges) are required because the setup script installs core components and adjusts server security gates.

DID YOU KNOW?
Container software (like Docker) adds translation layers that can delay incoming network webhooks by up to 5 milliseconds. Native Linux deployments remove these layers, making n8n react instantly to external automation signals.

4. Step-by-Step Installation Guide

Deploying n8n natively is a highly orchestrated procedure split across 11 discrete phases, managed by our automated installer script. Below is the operational sequence:

- **Phase 1 - System Checks:** Inspects package managers (apt, dnf, yum, pacman) and installs baseline networking utilities (curl, wget, gnupg2, and whiptail).
- **Phase 2 - Node.js Install:** Checks for Node.js. If missing or older than v18, automatically registers NodeSource v20 repositories and installs Node.js and NPM globally.
- **Phase 3 - Database Config:** If PostgreSQL is chosen, it installs packages, sets up database storage folders, creates a database named 'n8n', and assigns users secure passwords.
- **Phase 4 - NPM Install:** Installs PM2 process manager and the n8n application core globally on the machine via Node Package Manager (NPM).
- **Phase 5 - Config Environment:** Creates the configuration folder at /etc/n8n/ and writes the .env file. Generates a secure, random 64-character encryption key.
- **Phase 6 - PM2 Daemon setup:** Configures startup scripts, sets up the PM2 service, limits Node process memory, and activates automatic daily log rotation.
- **Phase 7 - Firewall configuration:** Finds the active security firewall (UFW, Firewalld, iptables) and opens the n8n port, HTTP port, and HTTPS port.
- **Phase 8 - Reverse Proxy Setup:** Installs Nginx, creates configuration files mapping web traffic, and starts the service.
- **Phase 9 - SSL Certbot:** Installs Let's Encrypt Certbot, verifies domain ownership, installs certificates, and configures secure HTTPS redirection.
- **Phase 10 - Helper scripts:** Creates helper scripts under /usr/local/bin for automated updates, daily backups, and clean uninstallation.
- **Phase 11 - Completion:** Purges installer temp files, validates that the PM2 daemon is running, and prints details.

5. Interactive Menu & Parameter Map

The installation script setup_n8n_native.sh uses whiptail, a console-based GUI dialog box system. It queries the administrator through graphic terminal dialog screens to avoid typos and validate network settings. Below is the mapping of all variables collected by the menus:

Collected Variable	Prompt Menu Type	Options & Default Values	Target Location
IP configuration	whiptail --menu	DHCP (Dynamic IP) / Static IP address	Netplan or nmcli settings
Database Backend	whiptail --menu	SQLite (File) / PostgreSQL (Daemon)	/etc/n8n/.env (DB_TYPE)
PostgreSQL Secrets	whiptail --passwordbox	PostgreSQL database admin password	/etc/n8n/.env (DB_PASSWORD)
Domain Name (FQDN)	whiptail --inputbox	Target web domain (e.g. n8n.domain.com)	/etc/nginx/sites-enabled/n8n
Server Listener Port	whiptail --inputbox	Default listening port (Default: 5678)	/etc/n8n/.env (N8N_PORT)
Enable daily backups	whiptail --yesno	Yes (Activates cron at 2 AM) / No	/etc/cron.d/n8n_backup

Future Screenshot Integration:

An image illustrating the interactive terminal selection box will be placed here to help visual learners orient themselves during initial boot.

6. Configuration Reference & Static IP Details

All configuration parameters for the native service are written inside a consolidated environment file at /etc/n8n/.env. This keeps database logins, encryption passwords, and custom ports secure.

Variable Name	Purpose & Context
N8N_PORT	Internal port where n8n listens (Default: 5678).
N8N_HOST	Listening IP. Set to 0.0.0.0 to accept traffic from all interfaces.
WEBHOOK_URL	Public URL used by external APIs (e.g. Slack) to trigger workflows.
N8N_ENCRYPTION_KEY	Generated key used to encrypt workflow credentials in the DB.
N8N_BASIC_AUTH_ACTIVE	Set to true to prompt users for credentials before loading n8n.
DB_TYPE	Database engine. Set to 'postgresdb' or defaults to SQLite.

Static IP Configuration Details

A Static IP address is an address that never changes. If your server's IP keeps changing (Dynamic IP), webhook integrations will break. The installer automates configuration across three platforms:

- **Debian/Ubuntu (Netplan):** Creates a configuration file at /etc/netplan/01-n8n-static.yaml, assigning static addresses, gateway, and nameservers. Runs 'netplan apply'.

- **RHEL/Fedora (NetworkManager):** Uses nmcli commands to adjust properties of the active network profile, setting IPv4 address, DNS, and manual IP assignment method.

- **Arch/Generic Fallback:** Uses direct 'ip addr' and 'ip route' commands to apply configurations immediately to the interface.

PRO TIP

If Netplan setup fails or you want to review configuration issues, you can run:

```
netplan --debug try
```

7. Database Backends & PM2 Process Management

Choosing the right database backend depends on how many workflows you plan to run. SQLite is simple and stores everything in a single file on disk. PostgreSQL runs as a standalone engine, perfect for high concurrency and heavy workflows.

Feature	SQLite Backend	PostgreSQL Backend
Storage Pattern	Single flat file (~/.n8n/database.sqlite)	Server process with access permissions
Concurrency	Locks database on write operations	Supports high concurrent query traffic
Performance	Fast for low-volume testing	Optimized for large workflow history logs
Configuration	Zero config, automated file creation	Requires host, port, DB user, & password

PM2 Process Daemon Configuration

PM2 is a process manager that runs in the background. It keeps n8n active constantly, restarting it if it experiences a memory issue or system restart.

- **Memory Limit Guard:** Configured via '--max-memory-restart 1G'. If n8n consumes more than 1GB of memory, PM2 will automatically restart it to keep the system healthy.
- **PM2 Log Rotation:** The setup configures 'pm2-logrotate' to keep server logs under 10 Megabytes (MB) per file, saving up to 7 historical logs to prevent filling up the system drive.
- **Boot Persistence:** Registers n8n as a systemd service so it boots automatically when the computer turns on.

8. Reverse Proxy, SSL & Security Best Practices

An Nginx reverse proxy routes incoming public requests from standard ports (80/443) to n8n's internal port (5678). This approach hides backend application ports, enables SSL encryption, and manages connections efficiently.

Nginx Proxy Configuration Block Details

Because n8n relies on WebSockets for real-time dashboard updates, the Nginx configuration must include specific proxy headers. Below is the Nginx server block template generated by the installer:

```
server {
    listen 80;
    server_name n8n.yourdomain.com;
    location / {
        proxy_pass http://127.0.0.1:5678;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_buffering off;
    }
}
```

Let's Encrypt SSL Integration

The installer script automatically downloads certbot and its Nginx plugin. Certbot validates the domain ownership, retrieves the TLS/SSL certificates, configures redirection blocks to force HTTPS, and schedules a systemd timer for automatic renewals.

Security Best Practices

- **Configuration Security:** Ensure /etc/n8n/.env is set to 'chmod 600' so only root or sudo users can read database credentials and encryption keys.

- **PM2 Execution Identity:** Run PM2 under a dedicated standard user account (using the SUDO_USER variable) rather than root, limiting potential damage in case of a vulnerability.

9. Backup, Recovery & Firewall Configuration

n8n stores workflow credentials, execution histories, custom workflows, and active tokens in its data directory (~/.n8n). The v3.0 installer sets up a backup system to prevent data loss.

Backup Mechanics & Retention Policies

- **Scheduled Execution:** Installs a script at /usr/local/bin/n8n_backup.sh that runs daily via cron (configured to run at 2 AM or your custom schedule).

- **Retention Management:** Archives the entire n8n database folder into a tar.gz file. It keeps only the last 7 daily archives, automatically deleting older files to manage disk space.

- **Restore Execution Workflow:** To restore a backup, stop the n8n daemon, extract the archive back to ~/.n8n, and restart PM2:

```
pm2 stop n8n
tar -xzf /var/backups/n8n/n8n_backup_[timestamp].tar.gz -C ~/.n8n/
pm2 start n8n
```

Firewall Configuration

The installer detects the active firewall utility and opens the necessary ports:

- **UFW (Ubuntu/Debian):** Allows n8n port (e.g. 5678) and web ports 80/443 (HTTP/HTTPS) if proxy is enabled.

- **Firewalld (Fedora/RHEL):** Adds runtime and permanent rules for port 5678, HTTP, and HTTPS services, then reloads the configuration.

- **iptables (Arch Linux):** Injects filter rules into the INPUT chain and uses netfilter-persistent to save configuration states.

10. System Debugging & Service Operations

This chapter lists critical operational commands for troubleshooting, process tracking, and database querying within the native environment:

PM2 Monitoring & Logging:

- **pm2 status / restart n8n:** Lists active Node.js processes or restarts the n8n core daemon process.
- **pm2 logs n8n / flush n8n:** Streams real-time server console output or flushes trailing service logs.
- **pm2 info n8n:** Exposes uptime, install directories, log locations, and active env variables.

Network & Database Diagnostics:

- **ss -tulpn | grep 5678:** Checks if n8n is listening on the expected port (or use netstat).
- **curl -I http://localhost:5678:** Tests the local n8n HTTP interface connection response directly.
- **systemctl status postgresql:** Validates that the database backend daemon is running.
- **sudo -u postgres psql -d n8n -c '\dt':** Exposes n8n table structures to verify database connectivity.

System Troubleshooting Actions:

- **Error EADDRINUSE:** Port conflict. Run 'fuser -k 5678/tcp' to kill blocking processes, or edit N8N_PORT in /etc/n8n/.env.
- **Node Environment Info:** Confirm baseline language dependencies via 'n8n --version' and 'node -v'.
- **DB Connection Refused:** Ensure the DB service is active and check the DB host/user credentials in /etc/n8n/.env.

11. FAQ, File Reference, & Uninstallation

Frequently Asked Questions (FAQ)

- **Q: Can I change database backends?:** Yes. Export workflows via the admin panel, change DB parameters in /etc/n8n/.env, and re-import.

- **Q: How do I upgrade n8n version?:** Execute 'sudo n8n_update.sh'. The script automatically stops the daemon, downloads the latest npm package, and restarts the engine.

Uninstalling the native setup

To cleanly remove the n8n application, dependencies, backup scripts, and cron logs, execute the uninstall script. Note that your user configuration directory (~/.n8n) containing workflow files is preserved to avoid data loss:

- **Uninstall Helper:** Execute 'sudo n8n_uninstall.sh' and follow the interactive prompt to purge the system configuration.

System File Reference Map

File / Path Location	Description & Purpose
/etc/n8n/.env	Central environment file containing ports and database credentials.
/usr/local/bin/start_n8n.sh	Startup script loaded by PM2 to export configuration variables.
/usr/local/bin/n8n_backup.sh	Daily archiving utility that backs up ~/.n8n/ to /var/backups/n8n/.
/usr/local/bin/n8n_update.sh	Helper script to update n8n to the latest version via NPM.
/usr/local/bin/n8n_uninstall.sh	Uninstallation script that cleans up configurations and Nginx blocks.